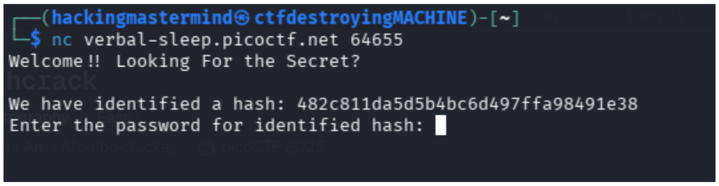
We are solving hashcrack, an easy-level cryptography challenge on cylab security academy. In this challenge, we must crack a secret message leaked by a server due to the admin using weakly hashed passwords.

I start off by connecting to the challenge instance via netcat and it asks for a password given a hash:



I knew I had to crack the hash to know the password, so I thought of using hashcat, a very well-known password cracking tool. Hashcat requires me to know the hashing algorithm used by the server to hash the password, so I used an online hash identifier tool to determine what it could be and it turns out to be either md5, md2, or md4 (I had a feeling it was most likely md5 since that hashing algorithm comes up a lot in these kinds of challenges):

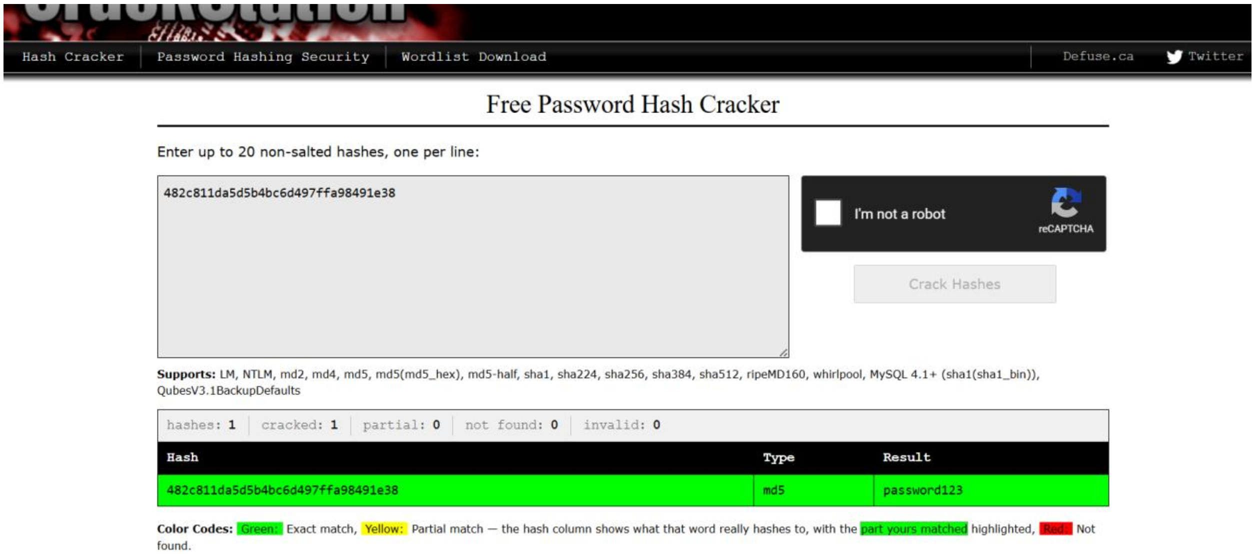
Now that I knew what hashing algorithm was being used, I was ready to use hashcat. To run hashcat, I need to specify the hashing algorithm used. Hashcat assigns numbers to each hashing algorithm. The number of the md5 algorithm is 0. I specify the attack mode as 0 using the -a option (use passwords from the dictionary file one after the other), the hash file (the file I put my hash in), and a dictionary file. However, when I tried to run it, it failed since I was missing some software:

```
┌──(hackingmastermind㉿ctfdestroyingMACHINE)-[~]
└─$ hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

clGetPlatformIDs(): CL_PLATFORM_NOT_FOUND_KHR

ATTENTION! No OpenCL, HIP or CUDA compatible platform found.

You are probably missing the OpenCL, CUDA or HIP runtime installation.

* AMD GPUs on Linux require this driver:
  "AMD Radeon Software for Linux" with "ROCm"
* Intel and AMD CPUs require this runtime:
  "Intel CPU Runtime for OpenCL" or PoCL
* Intel GPUs require this driver:
  "Intel Graphics Compute Runtime" aka NEO
* NVIDIA GPUs require this runtime and driver:
  "NVIDIA CUDA Toolkit" (both runtime and driver included)

Started: Fri Sep  4 09:33:42 2026
Stopped: Fri Sep  4 09:33:42 2026
```

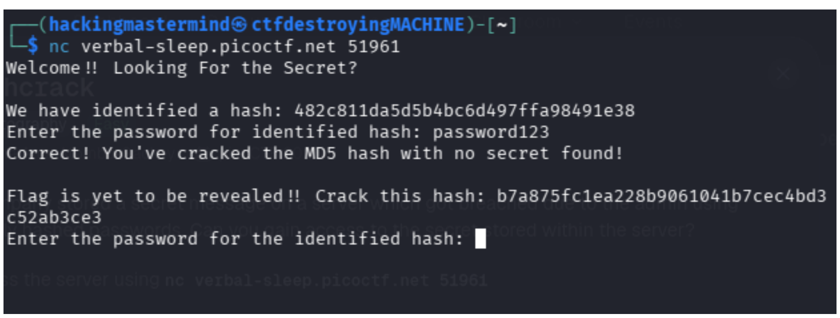
I tried to switch to john. John is a very well known password cracking tool as well. To use john, you specify the hash algorithm used (you don't necessarily need to but it makes things a lot easier), the dictionary file to attack the hash, and the hash file. However, john was taking too long to run:

```
┌──(hackingmastermind㉿ctfdestroyingMACHINE)-[~]
└─$ john --format=raw-md5 /usr/share/wordlists/rockyou.txt hash.txt
Warning: invalid UTF-8 seen reading /usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 53 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2
8×3])
Remaining 52 password hashes with no different salts
Warning: no OpenMP support for this hash type, consider --fork=2
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
Proceeding with incremental:ASCII
```

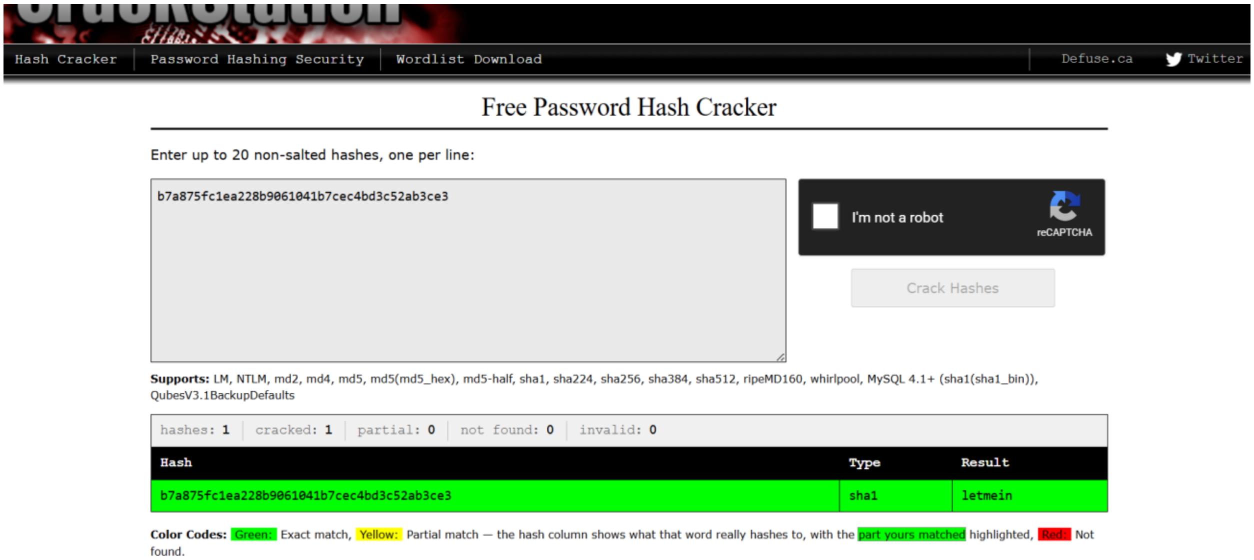
I finally decided to use an online hash cracking tool. The one I used was CrackStation. I pasted the hash and found the plaintext:



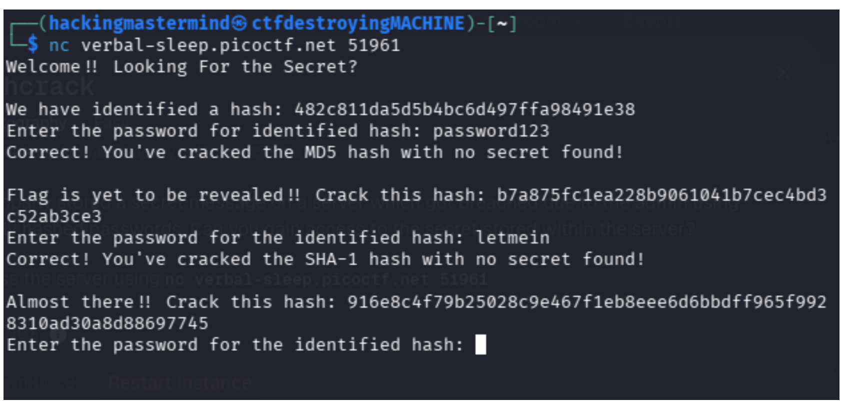
I then typed it into the program running on the challenge instance and was greeted with another hash to crack:



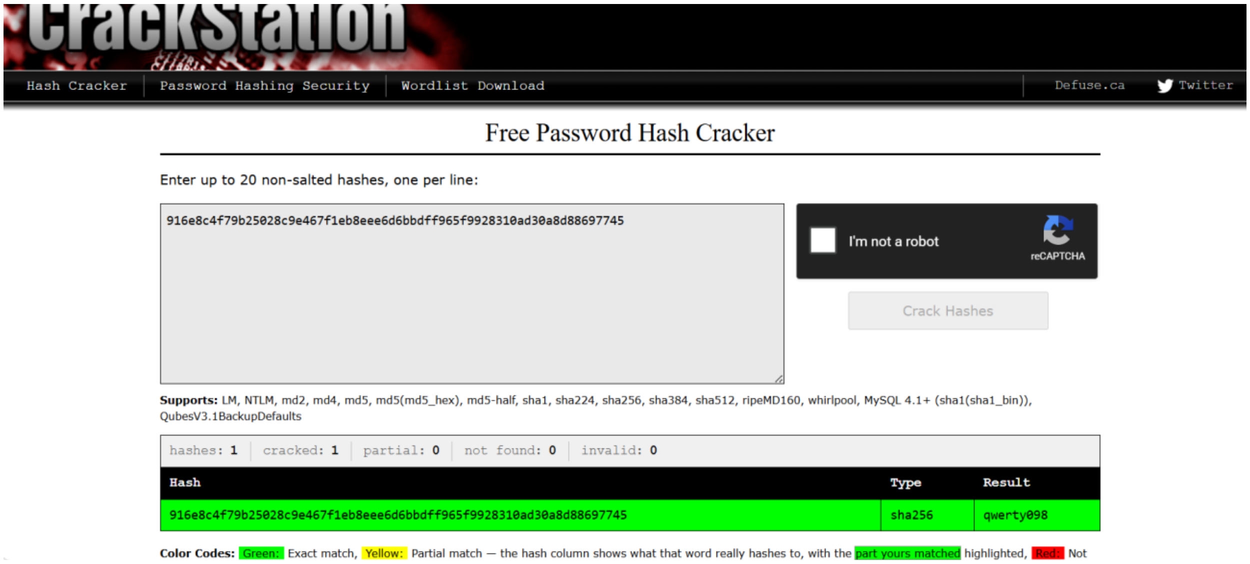
So, I also cracked this with Crackstation:



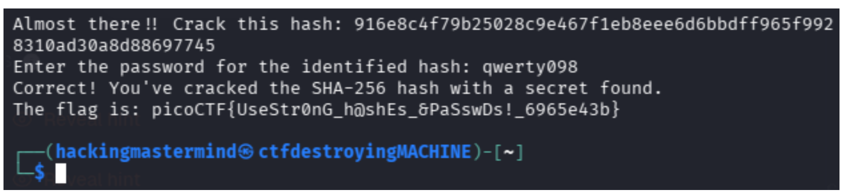
I then typed it into the program and was yet again greeted with another hash to crack:



I use Crackstation again to crack the hash:



I type it into the program and finally get the flag:



flag: picoCTF{UseStr0nG_h@shEs_&PaSswDs!_6965e43b}